

Synchronisation im Linux-Kernel

Primitiv, Kontext und Lebensdauer korrekt zusammenführen

2.08

Synchronisation schützt mehr als Speicherzugriffe

01

Exklusivität

Wer darf den Zustand gleichzeitig ändern?

02

Reihenfolge

Welche Writes müssen vor Reads sichtbar sein?

03

Lebensdauer

Wie lange bleibt ein Objekt gültig?

Mechanismus + dokumentierte Invariante + konsistente Nutzung = Vertrag

Fünf Fragen führen zum passenden Primitiv

Darf der Aufrufer schlafen?	ja: <code>mutex</code> / <code>rwsem</code>	nein: <code>spinlock</code> / <code>atomic</code>
Wird auf ein Ereignis gewartet?	<code>completion</code>	wait queue bei Zustandsbedingung
Ein Wert oder Invariante?	<code>atomic</code> / <code>bitops</code>	Lock für zusammengehörige Daten
Viele Leser, wenige Writer?	<code>rwsem</code> / <code>seqcount</code>	RCU bei passender Lebensdauer
CPU-lokaler Zustand?	<code>per-CPU</code> / <code>local_lock</code>	Aggregation statt globalem Lock

Der strengste Zugriffskontext bestimmt den zulässigen Mechanismus.

Mutexe schützen schlafbare kritische Sektionen

MUTEX

```
static DEFINE_MUTEX(config_lock);

int update_config(const struct config *new)
{
    int ret;
    mutex_lock(&config_lock);
    ret = validate_and_apply(new);
    mutex_unlock(&config_lock);
    return ret;
}
```

GEEIGNET

- Prozesskontext
- Workqueue
- Schlafende Funktion

VERBOTEN

- Hardirq / Softirq
- Spinlock gehalten
- IRQs deaktiviert

Ein Mutex passt nur in Kontexte, die schlafen dürfen.

Spinlocks schützen kurze atomare Abschnitte

SPINLOCK

```
static DEFINE_SPINLOCK(state_lock);  
static unsigned int state;  
  
void update_state(unsigned int next)  
{  
    spin_lock(&state_lock);  
    state = next;  
    spin_unlock(&state_lock);  
}
```

KURZ

- Wenige Operationen
- Keine langen Schleifen
- Kein I/O

KEIN SCHLAFEN

- Kein mutex_lock()
- Kein schedule()
- Keine blockierende API

Innerhalb des Spinlocks darf der Code nicht schlafen.

IRQ-Sharing verlangt passende Lock-Varianten

LOCK-VARIANTEN

```
unsigned long flags;  
spin_lock_irqsave(&queue_lock, flags);  
list_add_tail(&item->node, &queue);  
spin_unlock_irqrestore(&queue_lock, flags);  
  
spin_lock_bh(&net_lock);  
update_net_state();  
spin_unlock_bh(&net_lock);
```

01 Nur CPUs

spin_lock
spin_unlock

02 Mit Softirq

spin_lock_bh
spin_unlock_bh

03 Mit Hardirq

spin_lock_irqsave
spin_unlock_irqrestore

Alle Zugriffskontexte abdecken, aber nicht stärker sperren als nötig.

rwsem und rwlock erlauben parallele Leser

PROZESSKONTEXT

rw_semaphore

```
down_read() / up_read()  
down_write() / up_write()
```

Schlafbarer Kontext

SPINBASIERT

rwlock_t

```
read_lock() / read_unlock()  
write_lock() / write_unlock()
```

Atomarer Kontext

NUTZEN

Mehrere Leser dürfen gleichzeitig in die kritische Sektion.

Mehr Aufwand und mögliche Writer-Latenz. Zuerst messen.

Completions signalisieren Ereignisse

COMPLETION

```
static DECLARE_COMPLETION(done);

int start_and_wait(void)
{
    start_async_operation();
    wait_for_completion(&done);
    return result;
}

irqreturn_t handler(int irq, void *data)
{
    complete(&done);
    return IRQ_HANDLED;
}
```

01 complete()

Weckt einen Wartenden oder speichert eine Erfüllung.

02 complete_all()

Weckt alle Wartenden.
Reinitialisierung braucht einen klaren Lebenszyklus.

Wait Queues warten auf Bedingungen

WARTEN

```
DECLARE_WAIT_QUEUE_HEAD(data_wq);  
ret = wait_event_interruptible(  
    data_wq, data_ready);  
if (ret)  
    return ret;  
consume_data();
```

ZUSTAND ÄNDERN UND WECKEN

```
spin_lock(&data_lock);  
data_ready = true;  
spin_unlock(&data_lock);  
  
wake_up_interruptible(&data_wq);
```

Zustand ändern Lock lösen Wartende wecken

Die Bedingung ist die Wahrheit. Das Wake-up fordert nur eine erneute Prüfung an.

Atoms schützen einzelne Übergänge

ATOMIC UND BITOPS

```
static atomic_t pending = ATOMIC_INIT(0);  
atomic_inc(&pending);  
if (atomic_dec_and_test(&pending))  
    wake_up(&waitq);  
  
set_bit(READY_BIT, &flags);  
if (test_and_clear_bit(READY_BIT, &flags))  
    consume_ready();
```

GEEIGNET

- Ein Zustand
- Zähler
- Flag oder Bit
- Referenzzähler

NICHT AUSREICHEND

- Mehrere Schritte
- Check-then-act
- Mehrere Felder
- Komplexe Invariante

Atoms ersetzen keinen Schutz für zusammenhängenden Zustand.

Seqcount liefert konsistente Snapshots

LESER

```
unsigned int seq;
do {
    seq = read_seqcount_begin(&stats_seq);
    local_a = stats.a;
    local_b = stats.b;
} while (read_seqcount_retry(&stats_seq, seq));
```

WRITER

```
spin_lock(&stats_lock);
write_seqcount_begin(&stats_seq);
stats.a = next_a;
stats.b = next_b;
write_seqcount_end(&stats_seq);
spin_unlock(&stats_lock);
```

STARK

Kurze Writes und häufige Snapshot-Reads

Leser dürfen nicht auf ungültig werdende Pointer vertrauen. Writer-Kontext sorgfältig planen.

RCU trennt Lesen und Freigeben

LESER

```
rcu_read_lock();  
obj = rcu_dereference(current_obj);  
if (obj)  
    use(obj);  
rcu_read_unlock();
```

WRITER UND FREIGABE

```
old = rcu_dereference_protected(  
    current_obj, lockdep_is_held(&lock));  
rcu_assign_pointer(current_obj, new);  
synchronize_rcu();  
kfree(old);
```

01 Leser

Günstig und häufig

02 Writer

Neue Version veröffentlichen

03 Freigabe

Nach Grace Period

RCU verbindet Veröffentlichung und verzögerte Freigabe über einen klaren Lebenszyklus.

Acquire/Release ordnet lockfreie Übergaben

PUBLISHER

```
data = value;  
smp_store_release(&ready, true);
```

CONSUMER

```
if (smp_load_acquire(&ready))  
    use(data);
```

RELEASE

Frühere Writes stehen vor dem veröffentlichten Zustand.

ACQUIRE

Spätere Reads folgen nach dem beobachteten Zustand.

READ_ONCE() und WRITE_ONCE() allein bilden keine allgemeine Speicherbarriere.

Lock-Ordnung verhindert Deadlocks

PFAD A`lock(A) , danach lock(B)`**PFAD B**`lock(B) , danach lock(A)`**Zyklus: Deadlock****GLOBALE REGEL**`A immer vor B`**Lockdep: CONFIG_PROVE_LOCKING**

Locks nicht über Callbacks, copy_to/from_user oder unbekannte Funktionen halten.

Der Synchronisationsvertrag bleibt überprüfbar

KONTEXT	Darf der Pfad schlafen? Gibt es IRQ- oder Softirq-Sharing?
INVARIANTE	Welche Werte müssen gemeinsam konsistent bleiben?
MECHANISMUS	Passen API und Semantik zum Zugriffsmuster?
ORDNUNG	Welche Sichtbarkeit und Lock-Reihenfolge gilt?
LEBENSDAUER	Wann darf ein Objekt veröffentlicht und freigegeben werden?
PRÜFUNG	Lockdep, KCSAN, Stress und Review einsetzen

Einfach beginnen. Vertrag dokumentieren. Unter Last prüfen.